Two-Speaker Separation and Video-to-Audio Evaluation

Mrudduni Jitendra Modha

1. Two-speaker source separation

1.1 Approach and setup

I started from **SR-CorrNet**, a time-frequency speech separation model released in 2026 whose paper covers WSJ0-kMix, WHAMR! and LibriCSS but not Libri2Mix. I compared **three systems**. The first is the **official two-speaker checkpoint**, trained on WSJ0-2mix and **used without further training**. The second is the same checkpoint **fine-tuned on Libri2Mix** train-100. The third is a **three-layer BLSTM mask network trained from scratch** as a baseline. I wanted to know **how well a strong model transfers to a new corpus**, what fine-tuning adds, and what makes the remaining mixtures difficult.

SR-CorrNet's encoder makes a coarse split, and a weight-shared decoder then refines each speaker by estimating complex filters from time-frequency correlations. The network, its STFT layers and its weights come from the official repository, and SDR, STOI and PESQ come from mir_eval, pystoi and pesq. **Everything else is my own code**: the mixing, the waveform wrapper and chunked inference, the PIT SI-SDR loss, the training loop, the baseline, and the evaluation and analysis.

The mixtures are **built on the fly from LibriSpeech** using the official LibriMix metadata. I applied the same gains, resampling and truncation as the official script, at 8 kHz in min mode with clean speech, which matches the checkpoint's training data. Training follows the official recipe of PIT negative SI-SDR, a 0.5-weighted loss on intermediate decoder outputs and random 4 s crops. The one change is the **peak learning rate, lowered to 1e-4**.

Memory was the main obstacle. My first two Kaggle runs crashed. Measuring showed that SR-CorrNet keeps **about 4.2 GiB of activations per second of audio** per example, roughly 17 GiB for one 4 s crop. **Gradient checkpointing** cut this to **about 0.6 GiB**, and I checked on CPU that the gradients stayed bit-identical.

All runs used one GPU of Kaggle's 2 × T4 instance. Within its 1.25-hour budget, fine-tuning completed **one epoch of 150 updates**, at batch size 1 with 4-step gradient accumulation. The BLSTM trained for 100 epochs in 30 minutes. SR-CorrNet costs about 365 GFLOPs per second of audio, so I evaluated a **fixed random 500 of the 3,000 test mixtures**, covering all 40 test speakers. Model selection used 200 dev mixtures.

1.2 Results

Zero-shot, SR-CorrNet reached **18.87 dB SI-SDRi** (median 19.60). It raised STOI from 0.712 to 0.972, and PESQ from 1.61 to 3.75. **Fine-tuning took it to 20.05 dB** (median 20.74), with STOI 0.973 and PESQ 3.85. It also cut the share of mixtures below 5 dB from 1.4% to 0.6%.

The gain was broad. 93.8% of mixtures improved, by **1.18 dB on average** (95% bootstrap CI 1.01–1.35 dB). Female–male, female–female and male–male pairs gained similar amounts: 1.17, 1.07 and 1.33 dB. **Thirteen mixtures got worse** by more than 2 dB; I come back to them in Section 1.4.

The BLSTM reached 6.29 dB. It left 37% of mixtures below 5 dB and swapped speakers in 18% of them. I did not compute STOI or PESQ for it. Its dev score had flattened long before training ended. The 14 dB gap **mixes the effect of pretraining with differences in architecture and size**, 4.1 M against 13.6 M parameters.

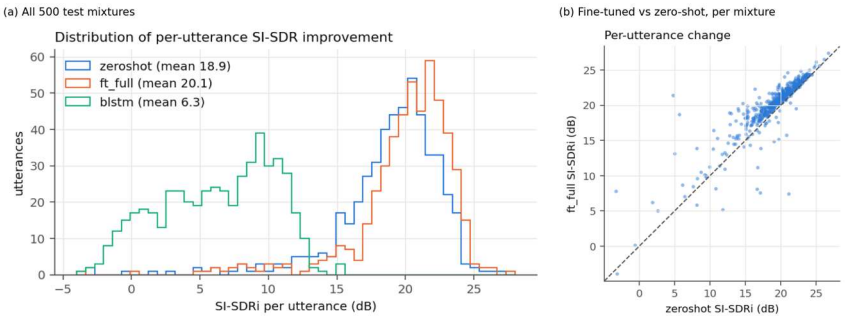


Figure 1. (a) Per-mixture SI-SDRi for the three systems. (b) Fine-tuned against zero-shot SI-SDRi; points above the diagonal improved.

1.3 Choice of metric

I report **SI-SDR improvement**. SI-SDR rescales the reference to best fit the estimate, then compares that fitted part with the residual, which holds both leakage from the other speaker and artifacts. I chose it because it is the **standard measure** for single-channel separation and the **training objective**, and changing the output level cannot inflate it. Subtracting the mixture's own score adjusts for mixtures that start out easier.

Its limits showed up in this study. Because it **trusts the reference**, noise inside a "clean" recording counts against the model, which is how a well-separated mixture in Section 1.4 scores −3.3 dB. It is also **not perceptual**: fine-tuning moved it by 1.2 dB, while STOI barely changed and PESQ rose by only 0.1.

The speaker assignment is chosen once per utterance, so **a swap partway through is largely hidden**. I added a **swap rate** over one-second windows to catch these, although its settings have not been checked against listeners. SI-SDR also punishes tiny time shifts that nobody would hear. For all these reasons I report medians and the low tail alongside the mean.

1.4 Failure analysis

I tagged each test mixture with five properties: the pitch gap between its speakers (the difference in median F0, estimated with YIN), their level difference, their overlap, the share of reference energy below 100 Hz, and the swap rate. For the zero-shot model, the strongest rank correlate of SI-SDRi was **low-frequency energy** ($\rho = -0.39$), followed by overlap (−0.33) and pitch gap (+0.22). Level difference hardly mattered. The **BLSTM tracked pitch gap** (0.59) and swap rate (−0.63) instead.

The clearest failure is a failure of the metric. The worst zero-shot mixture, 1580-141084-0024_2094-142345-0025, scores −3.3 dB, yet its STOI is 0.96 and its PESQ 3.5. One of its recordings has **72% of its energy below 100 Hz**, a rumble from the original LibriSpeech audio, and the model put that rumble into the other output (Figure 2). After a 100 Hz high-pass filter, the outputs score **17.3 and 13.0 dB**.

Eleven of the 500 mixtures have such a reference. They average 11.8 dB zero-shot against 19.0 dB for the rest, and they **gained 3.8 dB from fine-tuning** against 1.1 dB for the others. That hints at adaptation to LibriSpeech's recording conditions, though the sample is small. Reporting a **band-limited SI-SDR** next to the full-band score would keep this artifact apart from real errors.

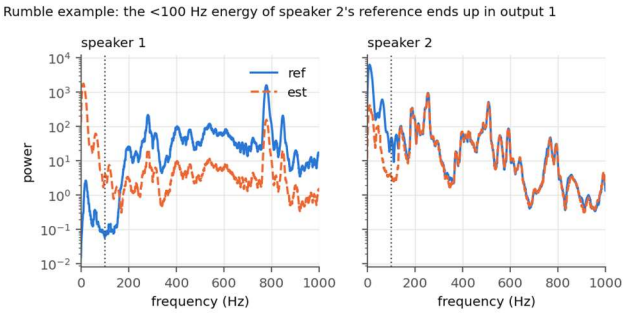


Figure 2. Zero-shot outputs for the rumble example. Below 100 Hz (dotted line), output 1 carries energy that belongs to speaker 2's reference.

The second failure is real. The worst mixture for every system, 3729-6852-0031_121-127105-0034, pairs **two female speakers whose median pitch differs by 0.04 Hz**. It scores −3.2 dB zero-shot and −3.9 dB fine-tuned. The outputs swap speakers in a quarter of the one-second windows, and output 2 keeps speaker 1 where speaker 2 is silent (Figure 3b).

Five of the seven worst zero-shot mixtures are female–female. SR-CorrNet's mean score rises from 17.6 dB for pitch gaps under 20 Hz to 19.8 dB above 100 Hz. The **BLSTM depends on pitch far more**, scoring 8.9 dB on mixed-sex pairs and 3.4–4.1 dB on same-sex ones. Without a pitch difference, a model has to rely on timbre and continuity, and utterance-level PIT does not penalize swaps within an utterance. **Segment-level PIT or a speaker-consistency loss** would target this.

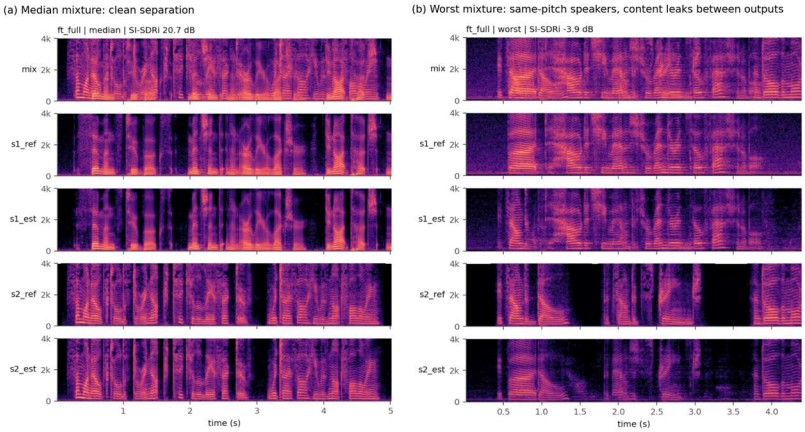


Figure 3. Fine-tuned outputs. (a) A median mixture, 20.7 dB. (b) The worst mixture, −3.9 dB: output 2 still contains speaker 1 around 1.4–1.8 s and 3.1–3.7 s, where speaker 2 is silent.

Third, fine-tuning made some mixtures worse by more than 2 dB, and **most of them are same-sex pairs**, against half of the test set. They are not swaps. The separation simply got less clean, with STOI falling from 0.967 to 0.947, and the largest drop is from 21.1 to 7.4 dB. **They also cluster by speaker.** Speaker 2830 appears in 35 of the 500 mixtures but in 5 of these 13, and mixtures with speaker 4970 gained nothing on average. My untested guess is that 150 noisy updates at batch size 1 pulled the model toward voices common in train-100.

1.5 Limitations

The main weakness is my fine-tuning schedule. Warm-up was 300 steps, but the budget allowed only 150, so the learning rate never reached its peak, and **1.18 dB is likely an underestimate**. Each system was **trained once**, so the confidence interval reflects test sampling only, and the evaluation covers 500 clean 8 kHz mixtures. Next I would train longer with a short warm-up and a larger batch. I would also freeze the encoder to locate the domain gap, and train SR-CorrNet from scratch to separate pretraining from architecture.

2. Evaluating audio generated from a silent video

A silent clip of a dog running and barking has many acceptable soundtracks, so **comparing generated audio with the original sample by sample would penalize good outputs**. I would call an output good if it could **plausibly be the real soundtrack**. It should sound natural and contain the right sources, barks and footsteps with no speech or music. Its sounds should line up in time with the action and match the number of events on screen. It should fit the place acoustically, and it should clearly depend on the video.

I would keep the **automatic metrics few, each answering one question**. **FAD on PaSST** embeddings measures overall realism, and **Meta's Audiobox Aesthetics** adds a per-clip quality score that FAD cannot give. The same **PaSST** model, used as a sound-class tagger, compares the classes present with the original audio and flags invented speech or music. **Synchformer's offset**, averaged over the first and last 4.8 s of each clip, measures synchronization. Its 0.2 s resolution catches gross errors but not those near the 0.125 s threshold people can detect, which the listening test covers. **ImageBind retrieval** tests whether each output identifies its own video among the 500, where chance is 0.2%. A recent benchmark found that FAD on PaSST and Synchformer's offset track human rankings.

Listeners are still needed for naturalness, for synchronization near the threshold of perception, for whether the space sounds right, and for the overall verdict. They are also needed to confirm that the metrics agree with people in this domain. **Typical failures** would include the wrong animal, human speech, or music and narration picked up from web videos. Others would be silence, timing offsets, wrong bark counts, room sound in an outdoor scene, a voice that changes between barks, vocoder artifacts, and outputs that ignore the video.

To compare three models on 500 videos, I would generate **three samples per video** from each model, normalize them to the same loudness, and count failed generations as failures. Every automatic metric would be computed on all outputs, with the original audio as an upper reference and audio from another video as a lower one. Per-video metrics would be compared with paired Wilcoxon tests and Holm correction, and FAD, which is computed over the whole set, with bootstrap confidence intervals over videos. For listening, I would take 100 videos stratified by scene type and sound sources and run **blind pairwise comparisons**, in which raters pick the better of two outputs for quality, synchronization, semantic fit and overall preference. Each pair would get five headphone-checked raters, with catch trials using a deliberately misaligned clip, and preferences would be fitted with a **Bradley–Terry model**. I would then check which automatic metrics agree with the listeners, and decide by a **rule fixed in advance**: semantic fit and synchronization as hard requirements, listener preference as the main outcome.

3. Two failure cases

Case A: the bark arrives 700 ms late. Viewers notice audio lagging by about 125 ms (ITU-R BT.1359), so 700 ms is far outside tolerance. I would detect it in **two independent ways**. One is a pretrained offset model such as **Synchformer**, after testing it on clips shifted by known amounts. The other is an **event check** that matches jaw-opening onsets from an animal pose model to bark onsets from a sound event detector.

The **cross-correlation peak** between the two event signals gives the lag, about +0.7 s here. Fitting the per-event offsets over time separates a **constant delay from drift and jitter**, and **onset F1** within the perceptual tolerance gives an interpretable score. Since a constant delay leaves every semantic metric untouched, **synchronization needs measures of its own**.

The **quickest fix** is to shift the audio back by the measured lag, which only works if the delay is constant, or to keep the best-synchronized of several samples. A **more lasting fix puts timing into the model**. A 24 fps video encoder with a small temporal transformer makes events like the jaw opening visible, and the audio reads them through **cross-attention**, with positions in seconds and each audio frame limited to video within about 150 ms. MMAudio does something

similar. Training should help too: a **contrastive loss against shifted audio** teaches timing, and out-of-sync clips should be removed from the data.

Case B: a park dog that sounds indoors. A small room adds a **reverberation tail** of roughly 0.3–0.6 s, dense early reflections and room tone. An open park has almost no tail, strong direct sound, and background such as wind or birds. Because barks are short, **the 50–300 ms after each bark shows the decay directly**. Blind estimates of RT60 and the direct-to-reverberant ratio, plus ambience tags on the bark-free segments, add evidence. The video supplies the expectation, through a scene classifier, sky and depth cues, and the dog's distance.

Across a large dataset I would report three things. The first is **how often outdoor videos receive indoor-sounding audio**. The second is the **correlation between reverberation predicted from the image and reverberation measured in the audio**. The third is **FAD per scene type**, since a single pooled FAD cannot see this error. Before trusting any of these, I would **validate the detectors** on outdoor clips with real room impulse responses applied, so the mismatch is known.

4. Use of AI tools

Claude Code did most of the implementation. It wrote the data pipeline, the training and evaluation scripts, the analysis code and the Kaggle notebook generator, and it ran the CPU smoke tests. It also helped draft this report, which I then edited.

I chose to fine-tune SR-CorrNet on Libri2Mix, downloaded LibriSpeech and the LibriMix metadata, and set the 3–4 hour budget on free Kaggle T4 GPUs. That budget shaped both the test subset and the time-limited training. **I ran every experiment on Kaggle**, a pipeline check and three training runs, and collected the logs and results. The **CUDA traceback I retrieved** from the second run located the out-of-memory failure, which led to gradient checkpointing.

After one successful fine-tuning run, I **decided to stop training** and spend the remaining time on analysis. I asked for a **critical review of the methodology** and chose which issues to fix, including the confound between pretraining and architecture and the warm-up misconfiguration. I also reviewed the results, chose the figures and edited this report. I **worked through the Part 2 and 3 answers myself**, using AI to draft paragraphs and consolidate information.

Three problems needed real debugging. The first run's "fine-tuned" scores equaled zero-shot on every mixture, because training had crashed in its first epoch and the epoch-0 checkpoint had been evaluated. The local data turned out to be raw LibriSpeech rather than Libri2Mix, so the official mixing had to be reimplemented. And an early test subset covered only 8 of the 40 test speakers.